

Session 05 - Interface, Access Modifiers, Abstract Class

Naveen AutomationLabs | 2026 Playwright Batch G1

1. Interface - Basics

An interface defines a blueprint or contract. It specifies what properties and methods an object or class must have, without providing the implementation.

- You cannot create an object of an interface directly.
- Interfaces only have method declarations -- no method body, no business logic.
- Interfaces define the shape of objects and the contract for classes.

2. Interface with Objects

An interface can define the structure of an object -- what properties and methods it must contain.

```
interface User {
  name: string;
  age: number;
  salary?: number; // optional property
  readonly compName: string; // cannot be changed after creation
  city: string;
  coding(): void;
  calcSal(): number;
}

let u1: User = {
  name: 'Gaurav',
  age: 30,
  compName: 'IBM',
  salary: 12.33,
  city: 'LA',
  coding(): void {
    console.log(this.name + ' is coding');
  },
  calcSal(): number {
    return 100;
  }
};

console.log(u1);
u1.coding(); // Gaurav is coding
console.log(u1.calcSal()); // 100
```

The ? makes salary optional. readonly means compName can be set once during creation but never changed after that.

3. Access Modifiers - public, private, protected

Access modifiers control where a class member (property or method) can be accessed from.

- public: accessible everywhere -- inside the class, child classes, and outside the class. This is the default.
- private: accessible only inside the same class. Not in child classes, not outside.
- protected: accessible inside the class and in child classes. Not outside the class.

```
class User {
  public testing(): void {
    console.log('testing method');
  }

  private coding(): void {
    console.log('coding');
  }

  public doCoding(): void {
    this.coding(); // private method called inside same class - OK
  }

  protected management(): void {
    console.log('management');
  }
}

class Employee extends User {
  public working(): void {
    this.testing(); // public - OK
    this.management(); // protected - OK in child class
    this.doCoding(); // public - OK
    // this.coding(); // Error: private - NOT accessible
  }
}

// Outside the class:
let u1: User = new User();
u1.testing(); // public - OK
// u1.coding(); // Error: private
// u1.management(); // Error: protected
```

Use private for internal helper methods. Use protected for methods that child classes need but external code should not access. Use public for the external API.

4. Abstract Classes

An abstract class is a class that cannot be instantiated (no object creation). It can have both abstract methods (no body) and non-abstract methods (with body). This gives you partial abstraction -- 0% to 100%.

- Abstract methods: only declaration, no body. Child class MUST provide the implementation.
- Non-abstract methods: have a body. Child class can use as-is or override them.

```
abstract class Page {
  constructor() {
    console.log('default page const....');
  }

  // Abstract methods: child MUST implement these
  abstract title(): void;
  abstract url(): void;

  // Non-abstract methods: child can use or override
  pageLoading(): void {
    console.log('page is loaded in 20 secs');
  }
}
```

```

footers(): void {
console.log('display footer links');
}
}

class LoginPage extends Page {
constructor() {
super(); // must call parent constructor
console.log('Login page const....');
}

override title(): void {
console.log('Google Login');
}
override url(): void {
console.log('https://google.com/login.html');
}
override pageLoading(): void {
console.log('Login page is loaded in 5 secs');
}

// Own method:
doLogin(username: string, password: string): void {
console.log(username, password, 'logged in to the app');
}
}

let lp: LoginPage = new LoginPage();
lp.title(); // Google Login
lp.url(); // https://google.com/login.html
lp.pageLoading(); // Login page is loaded in 5 secs
lp.doLogin('naveen', 'naveen@123');

```

5. Interface with Classes (implements)

A class uses the 'implements' keyword to follow an interface contract. The class must provide the body for every method declared in the interface. Interfaces give you 100% abstraction.

Interface inheritance:

```

interface WHO {
covidVaccination(): void;
}

interface USMedical extends WHO {
minFee: number;
physio(): void;
cardio(): void;
emergency(): void;
}

interface UKMedical extends WHO {
eNT(): void;
oncology(): void;
emergency(): void;
}

interface IndianMedical extends WHO {
pedia(): void;
}

```

```

dental(): void;
gastro(): void;
emergency(): void;
}

```

Class implementing multiple interfaces:

```

class FortisHospital implements USMedical, UKMedical, IndianMedical {
  minFee: number = 10;

  covidVaccination(): void {
    console.log('FH -- covidVaccination');
  }
  emergency(): void {
    console.log('FH -- emergency');
  }
  physio(): void { console.log('FH -- physio'); }
  cardio(): void { console.log('FH -- cardio'); }
  eNT(): void { console.log('FH -- eNT'); }
  oncology(): void { console.log('FH -- oncology'); }
  pedia(): void { console.log('FH -- pedia'); }
  dental(): void { console.log('FH -- dental'); }
  gastro(): void { console.log('FH -- gastro'); }

  // Own method (not from any interface):
  medicalTest(): void {
    console.log('FH -- medical testing');
  }
}

let fh: FortisHospital = new FortisHospital();
fh.cardio();
fh.emergency();
fh.medicalTest();
fh.covidVaccination();

```

6. Top Casting with Interfaces

Just like with classes, a child class object can be referred to by a parent interface reference variable.

```

// Full access with class type:
let fh: FortisHospital = new FortisHospital();
fh.cardio(); // OK
fh.dental(); // OK
fh.medicalTest(); // OK

// Limited access with interface type (top casting):
let us: USMedical = new FortisHospital();
us.cardio(); // OK (defined in USMedical)
us.physio(); // OK (defined in USMedical)
us.emergency(); // OK (defined in USMedical)
// us.dental(); // Error: not in USMedical interface

let uk: UKMedical = new FortisHospital();
// uk.cardio(); // Error: not in UKMedical

```

Top casting limits what methods are visible based on the reference type. The object still has all methods, but you can only access the ones defined in the interface you are using.

7. Abstract Class vs Interface - Quick Comparison

-
- Abstract class: can have both abstract and non-abstract methods. Partial abstraction (0-100%).
 - Interface: all methods are abstract (no body). 100% abstraction.
 - A class can extend only ONE abstract class (single inheritance).
 - A class can implement MULTIPLE interfaces.
 - Abstract class can have constructors. Interfaces cannot.
 - Use abstract class when child classes share some common code. Use interface when you only need a contract.